Foundation of Cryptography, Lecture 11
Secure Computation, *Many* Parties[1]

Handout Mode

Iftach Haitner

Tel Aviv University.

January 08, 2026

[1] Prepared: 2026/01/07,10:32:07.

Section 1

The Model

Real-model execution

For protocol $\pi = (\mathsf{P}_1, \ldots, \mathsf{P}_n)$ and inputs $x_1, \ldots, x_n \in \{0,1\}^*$, let $\mathsf{REAL}_\pi(x_1, \ldots, x_n)$ be the joint output of $(\mathsf{P}_1(x_1), \ldots, \mathsf{P}_n(x_n))$.

Given a protocol π, algorithm taking the role of one of the parties in π is:

- Honest — acts exactly according to π.
- Semi-honest — acts according to π, but might output **additional information**.
- Malicious — acts arbitrarily.

Dishonest parties (semi-honest or malicious) can cooperate

Ideal model execution

For oracle-aided algorithms $\widehat{\pi} = (\widehat{\mathsf{P}}_1, \ldots, \widehat{\mathsf{P}}_n)$, inputs $x_1, \ldots, x_n \in \{0,1\}^*$ and a function $f = (f_1, \ldots, f_n)$, let $\mathsf{IDEAL}^f_{\widehat{\pi}}(x_1, \ldots, x_n)$ denote the joint output of the parties in the end of the following experiment:

1. The input of $\widehat{\mathsf{P}}_i$ is x_i.
2. $\widehat{\mathsf{P}}_i$ sends some y_i to the trusted party .
3. Trusted party sends $z_i = f_i(y_1, \ldots, y_n)$ to $\widehat{\mathsf{P}}_i$.
   - (In today's talk) No option to abort the trusted party.
4. Each party outputs some value.

An oracle-aided algorithm is:

- Honest — sends its private input to the trusted party (i.e., sets $y_i = x_i$), and its only output is the value it gets from the trusted party (i.e., z_i).
- Semi-honest, sends its input to the trusted party, outputs z_i plus possibly additional information.
- Malicious — acts arbitrarily.

Dishonest parties (semi-honest or malicious) can cooperate

Secure computation

Definition 1 (Secure computation)

A PPT protocol $\pi = (\mathsf{P}_1, \ldots, \mathsf{P}_n)$ t-securely computes f, if $\forall$ PPT protocol $\pi' = (\mathsf{P}'_1, \ldots, \mathsf{P}'_n)$ with at most t dishonest parties (with respect to π), exists oracle-aided protocol $\widehat{\pi} = (\widehat{\mathsf{P}}_1, \ldots, \widehat{\mathsf{P}}_n)$, such that s.t.

$$\{\mathsf{REAL}_{\pi'}(\mathbf{x})\}_{\mathbf{x}\in(\{0,1\}^*)^n} \approx_c \{\mathsf{IDEAL}^f_{\widehat{\pi}}(\mathbf{x})\}_{\mathbf{x}\in(\{0,1\}^*)^n}$$

If P'_i is honest, then $\widehat{\mathsf{P}}_i$ is honest. If π' is all-honest, the ensembles are identical.

Part I

BGW Semi-Honest Protocol

Semi-honest protocol

For any poly-time f, we present protocol for f that is $(t < n/2)$-secure against semi-honest adversaries.

- Achieves perfect security: the distribution are identical.
- Can be compiled into computationally secure against malicious adversaries (using the compiler we seen last talk).
- Or, using more sophisticated means, can be "compiled" to achieve perfect security against malicious adversaries (next part).

Section 2

Secret Sharing

Secret sharing

A randomized mechanism that allows splitting shares of a secret between a set of parties, such that only authorized set of parties can reconstruct the secret from their shares.

- A fundamental tool in cryptography.
- Let n denote the number of parties
- Let $P(n)$ be the family of all subset of $[n]$, not including the empty set.
- Access structure is simply a set $\mathcal{A} \subseteq P(n)$
- We focus on monotone A: $\mathcal{I} \in \mathsf{A} \implies \mathcal{I}' \in \mathcal{A}$ for any $\mathcal{I}' \supseteq \mathcal{I}$
- $\mathcal{A} = \{\mathcal{I} \subseteq [n] \colon |\mathcal{I}| \geq t\}$ is called the t-out-of-n.

Secret sharing schemes

Definition 2 (Secret sharing schemes)

A secret sharing scheme over domain $\mathcal{D}$ for access structure $\mathcal{A} \subseteq P(n)$, is a pair of algorithms (E, D) such that the following holds:

Correctness: $\forall x \in \mathcal{D}$:

$$\mathsf{E}(x) \text{ outputs } z \in \Sigma^n, \text{ and } \mathsf{D}(\mathcal{I}, \mathsf{E}(x)) = x \text{ for any } \mathcal{I} \in \mathcal{A}.$$

Secrecy: $\forall x, x' \in \mathcal{D}$ and $\mathcal{I} \notin \mathcal{A}$: $\mathsf{E}(x)_{\mathcal{I}} \equiv \mathsf{E}(x')_{\mathcal{I}}$

- E is randomized
- We focus on efficient (poly-time) schemes for t-out-of-n access structure
- Example: scheme for the n-out-of-n access structure over $\mathcal{D} = \{0, 1\}$:
  - $\mathsf{E}(x)$: outputs $z \xleftarrow{\mathsf{R}} \{0, 1\}^n$ such that $\bigoplus_{i \in [n]} z_i = b$
  - $\mathsf{D}_{[n]}(z)$: outputs $\bigoplus_i z_i$

Polynomial over a finite field

Fix finite field $\mathbb{F}$ of size at least $n+1$, and identify $n+1$ values in $\mathbb{F}$ with $\{0,1,\ldots,n\}$.

- We identify a polynomial $p \in \mathbb{F}[x]$ with its coefficient: i.e., $p(x) = \sum_{i=0}^{k} \alpha_i x^i$ is identifies by $(\alpha_0,\ldots,\alpha_k)$.
- p is of degree (at most) d, if $\alpha_i = 0$ for all $i > d$.
- Sampling a random degree d polynomial p with $p(0) = s$, can be done by setting $\alpha_0 \leftarrow s$, and $(\alpha_1,\ldots,\alpha_d) \xleftarrow{R} \mathbb{F}^d$.
- Addition: Let p, p' be two degree $d < n$ polynomial over $\mathbb{F}$, and let $z, z' \in \mathbb{F}^n$ be their evaluations over $[n]$.

  Then $z + z'$, point-wise addition, is the evaluations values of the degree d polynomial $q = p + p'$ with $q(0) = p(0) + p'(0)$.
- Multiplication: Let p, p' be two degree $d < n/2$ polynomial over $\mathbb{F}$, and let $z, z' \in \mathbb{F}^n$ be their evaluation over $[n]$.

  Then $z * z'$, point-wise multiplication, is the evaluations values of the degree $2d$ polynomial $q = p \cdot p'$ with $q(0) = p(0) \cdot p'(0)$.

Polynomial interpolation

- Let p be a degree d polynomial over $\mathbb{F}$.
- Let $\mathcal{I} \subseteq [n]$ be of size $d+1$.
- There exists a single degree d polynomial p' with $p'(i) = p(i)$ for all $i \in \mathcal{I}$.
- p can be reconstructed efficiently from any $\{(i, p(i))\}_{i \in \mathcal{I}}$, e.g., *Lagrange interpolation*.
- $\exists \lambda_0, \ldots, \lambda_n \in \mathbb{F}$ (same for all t) so that $p(0) = \sum_{i=1}^{n} \lambda_i \cdot p(i)$.

$d+1$ point are necessary:

- If p is sampled uniformly (over degree d polynomials), given $\{(i, p(i))\}_{i \in \mathcal{I}}$ for $\mathcal{I} \subseteq [n]$ of size $< d+1$, the value of $p(0)$ is uniform over $\mathbb{F}$.

Shamir t-out-of-n scheme over $\mathbb{F}$

Algorithm 3 (Shamir t-out-of-n)

- $\mathsf{E}^t(x)$:
  1. Sample random polynomial p of degree $t-1$ such that $p(0) = x$
  2. Output $(p(1), \ldots, p(n))$.
- $\mathsf{D}^t_{\mathcal{I}}(z)$ for t-size $\mathcal{I} \subset [n]$, and $z \in \mathbb{F}^t$:
  1. Reconstruct (e.g., using interpolation) $s \leftarrow p(0)$
  2. Output s.

- Correctness: clear.
- Secrecy: for any $\mathcal{I} \subset [n]$, of size $< t$, the value of $p(0)$ is uniform in $\mathbb{F}$, condition on $p(i) = z_i$ for all $i \in \mathcal{I}$.

Section 3

The Protocol

The settings

- Fix $n \in \mathbb{N}$, $t < n/2$ and finite field $\mathbb{F}$ with $|\mathbb{F}| \geq n + 1$.
- Arithmetic circuit: gates, wires, inputs, outputs, values, computation
- Fix a single output functionality $f \colon \mathbb{F}^n \mapsto \mathbb{F}$ computed by a poly-size arithmetic circuit C.

  We later discuss f computed by Boolean circuit

- We present perfectly secure protocol for computing f, against t semi-honest adversaries
- Additions and multiplications are over $\mathbb{F}$

The protocol

Protocol 4 ($\pi = (\mathsf{P}_1, \ldots, \mathsf{P}_n)$. P_i's input: $x_i \in \mathbb{F}$.)

1. The parties computes, in bottom-up order, shares for all wires of C:
   - **1.1** Input wires:
     - **1.1.1** P_i: Sample $z_i \xleftarrow{\mathsf{R}} \mathsf{E}^{t+1}(x_i)$ and send $(z_i)_j$ to P_j.
     - **1.1.2** P_i: Set share of the j'th input wire to $(z_j)_i$.
   - **1.2** Addition gates: P_i: set share of the output wire to $a_i + b_i$, for a_i and b_i being its shares of the gate's input wires.
   - **1.3** Multiplication gates: next slide
2. Each P_i broadcast its share o_i of the (single) output wire.
3. Each party output $\mathsf{D}^{t+1}_{[n]}(o_1, \ldots, o_n)$

Claim 5 (Implies correctness of the protocol)

Let v be the value $\mathsf{C}(x_1, \ldots, x_n)$ assigns to a given wire, and let $z_1, \ldots, z_n$ be the value the above (random) protocol assign to the parties' shares of this wire. Then $\mathsf{D}^{t+1}_{[n]}((z_1, \ldots, z_n)) = v$.

Multiplication gates

Protocol 6 (Mult. P_i holding shares a_i, b_i.)

1. Let $(d_{i,1}, \ldots, d_{i,n}) \xleftarrow{R} E^{t+1}(c_i \leftarrow a_i \cdot b_i)$.
2. Send $d_{i,j}$ to P_j.
3. Output $o_i \leftarrow \sum_{j=1}^{n} \lambda_j \cdot d_{j,i}$

Claim 7

Mult securely computes the functionality that on input $(a_1, b_1), \ldots, (a_n, b_n)$, sends $E^{t+1}(D^{t+1}_{[n]}(a_1, \ldots, a_n) \cdot D^{t+1}_{[n]}(b_1, \ldots, b_n))_i$ to P_i.

Proof: Correctness

$$\sum_i \lambda_i o_i = \sum_i \lambda_i \sum_j \lambda_j d_{j,i} = \sum_j \lambda_j \sum_i \lambda_i d_{j,i} = \sum_j \lambda_j c_j = a \cdot b$$

Security?

Claim 8

Protocol 4 perfectly computes f against t semi-honest adversaries

Proof: Assume $\mathsf{P}_1, \ldots, \mathsf{P}_t$ are corrupted.

Algorithm 9 ($\widehat{\mathsf{P}}_i$ on input x_i)

1. Send x_i to the trusted party, and let o be its answer.
2. Jointly with the other corrupted parties, emulate the first step of $(\mathsf{P}_1(x_1), \ldots, \mathsf{P}_t(x_t), \mathsf{P}_{t+1}(0), \ldots, \mathsf{P}_n(0))$.
3. Choose the value of $o_{t+1}, \ldots, o_n$ uniformly at random conditioned on $\mathsf{D}_{[n]}^{t+1}(o_1, \ldots, o_n) = o$
4. Complete the execution of P_i, and output what it does.

Main observation: Wires' shares seen by corrupted parties have same distribution in real and ideal experiment.

Proof: Clear for *input wires*. For *gates output wires*, proof by induction on the gate *depth*.

1. Addition gates: local computation
2. Multiplication gates: Mult securely compute multiplication functionality

Extensions

- Have we used the fact that $t < n/2$?
- $t = n - 1$? Use OT-based computational $(n-1)$-secure Mult protocol.
- Boolean f?
  1. Input mapping: $0 \mapsto 0$, $1 \mapsto 1$.
  2. $\neg a \mapsto 1 - a$
  3. $a \wedge b \mapsto a \cdot b$
  4. $a \vee b \mapsto a + b - a \cdot b$

Part II

Malicious BGW

What might go wrong?

What goes wrong in the semi-honest protocol if the corrupt parties can act arbitrarily?

1. Input sharing
2. Multiplication
3. Output reconstruction

We present (perfect) solution for $t < n/3$ (which is tight for unbounded adversaries)

Verifiable secret sharing

1. Sharing phase is a protocol led by the dealer.
2. It has no way to cheat.
3. For $t < n/3$, there is VSS for Shamir t-out-of-n SS scheme.

   See description in this video
4. Using VSS, solves the sharing parts.

Shamir SS is Error Correction Code

- Error Correction Codes (ECC) enables encoding a message m into a longer one $c \leftarrow \mathrm{E}(m)$, the *codeword*, such that m be be recovered from a noisy variant $\widetilde{c}$ of c.
- Shamir t-out-of-n SS scheme is an ECC that can recover up to $(n-t)/2$ errors
- For $n \geq 3t+1$, Shamir can recover from t errors.
- Solves the reconstruction parts

Secure multiplication protocol

- Idea: Add a consistency-check sub-protocol to Mult, to enforce P_i to share the correct value of c_i.
- Problem: What if the parties cheat in the consistency-check sub-protocol?
- Solution: we use a linear (addition only) protocol, and thus can be compiled using the malicious BGW protocol.